

PolyGet: Driver-Based Package Search Implementation Plan

For Antigravity: Implement task-by-task, in order. This plan is independent of the "Manager Store, Discovery Catalog, and Repos Tab" plan, but both touch `app/core/manager.py` — if that plan has already been partially implemented, merge base-class additions rather than overwriting.

Goal: The package Store's search currently lives entirely inside `SearchWorker` (`app/ui/main_window.py`), as five hardcoded async functions, one per manager. This means (1) any future manager added to the catalog needs a manual UI-thread edit to ever show search results, (2) a real bug — Flatpak's search call uses a `-j / --json` flag that doesn't exist on the `flatpak search` subcommand — has gone unnoticed because the failure is silent, and (3) Pipx's "search" is actually just an exact-name PyPI lookup, not a real search, and can't return partial matches.

Architecture:

1. Add `search_packages(query)` to the `PackageManager` base class, defaulting to `NotImplementedError`, following the same interface pattern as `check_updates()` / `list_installed()`.

2. Move each manager's search logic out of `SearchWorker` and into its own driver file, fixing the Flatpak flag bug in the process.
3. Refactor `SearchWorker` to loop over `discover_managers()` and call `search_packages()` on whichever ones implement it, instead of five hardcoded functions — this is what makes future catalog managers (pacman, apt, zypper, etc.) automatically searchable once someone implements the method, with zero UI changes.
4. Give Pipx a real local-index-based search instead of an exact-name lookup, since PyPI has no live search API to call.

Tech Stack: Python 3, PySide6, asyncio (unchanged — no new dependencies except optional local cache file for Pipx)

Task 1: Add `search_packages()` to the Base Class

Files:

- Modify: `app/core/manager.py`

Step 1: Add the method

```
async def search_packages(self, query:
str) -> list[dict[str, Any]]:
    """Search this manager's package
    source for a query string.
```

Returns:

```
list[dict[str, Any]]: Each dict
should have at minimum 'name', 'id',
                        'description', 'version'.
Callers should add 'source' themselves.

Raises:
    NotImplementedError: If this
manager doesn't support searching.
"""
    raise NotImplementedError(f"
{self.name} does not support package search")
```

Step 2: Verify

Confirm the five existing manager classes still import and instantiate cleanly (this method isn't implemented on them yet — that's Task 2).

Task 2: Migrate DNF, Cargo, NPM Search Logic Into Their Drivers

Files:

- Modify: `app/core/drivers/dnf.py`
- Modify: `app/core/drivers/cargo.py`
- Modify: `app/core/drivers/npm.py`
- Modify: `tests/test_dnf_driver.py`, create
`tests/test_cargo_driver.py`, create
`tests/test_npm_driver_search.py`

Step 1: DNF — move the existing regex-based search
from `SearchWorker.search_dnf()`

```

        async def search_packages(self, query:
str) -> list[dict[str, Any]]:
        try:
            proc = await
asyncio.create_subprocess_exec(
                "dnf", "search", query,

stdout=asyncio.subprocess.PIPE,
stderr=asyncio.subprocess.PIPE
            )
            stdout, _ = await
asyncio.wait_for(proc.communicate(),
timeout=12.0)
            import re
            pattern = re.compile(r'^\s*([a-zA-
Z0-9\-\_+.]+)\.([a-zA-Z0-9_]+\s+(.+)$')
            results = []
            for line in
stdout.decode(errors="ignore").splitlines():
                match = pattern.match(line)
                if match:
                    pkg_name = match.group(1)
                    if any(x in pkg_name for x
in ("-debuginfo", "-debugsource", ".src")):
                        continue
                    results.append({"name":
pkg_name, "id": pkg_name,

"description": match.group(3), "version": ""})
            return results
        except Exception:
            return []

```

Step 2: Cargo — move `search_cargo()` logic verbatim
(the TOML-line regex parsing), same signature, returning

`version` populated this time since cargo's output includes it (unlike DNF's plain search).

Step 3: NPM — move `search_npm()` **logic verbatim** (the `npm search --json` call), same signature.

Step 4: Delete the corresponding functions from `SearchWorker` once each driver's version is verified working in isolation (don't delete all at once — verify one, then move to the next, so a regression is easy to bisect).

Step 5: Tests

For each driver, mock `create_subprocess_exec` to return canned stdout matching real command output, assert `search_packages()` parses correctly, and assert it returns `[]` (not an exception) on malformed or empty output.

Task 3: Fix and Migrate Flatpak Search

Files:

- Modify: `app/core/drivers/flatpak.py`
- Modify: `tests/test_drivers_installed.py` or create `tests/test_flatpak_search.py`

Step 1: Replace the broken `-j` /JSON approach with `--columns`

Flatpak's `search` subcommand only supports `--arch` and `--columns` (confirmed against upstream `flatpak-builtins-search.c` — there is no `-j` / `--json` entry for

`search`, unlike `list` or `remote-list` which do have it).
Use tab-separated columns instead:

```
    async def search_packages(self, query:
str) -> list[dict[str, Any]]:
    try:
        proc = await
asyncio.create_subprocess_exec(
            "flatpak", "search", "--
columns=name,description,application,version,r
emotes", query,

stdout=asyncio.subprocess.PIPE,
stderr=asyncio.subprocess.PIPE
        )
        stdout, _ = await
asyncio.wait_for(proc.communicate(),
timeout=10.0)
        results = []
        for line in
stdout.decode(errors="ignore").splitlines():
            parts = line.split("\t")
            if len(parts) >= 5:
                results.append({
                    "name": parts[0],
                    "description": parts[1], "id": parts[2],
                    "version": parts[3],
                    "remote": parts[4].split(",")[0]
                })
        return results
    except Exception:
        return []
```

**Step 2: Sanity-check on a real system before assuming
this is the only issue**

Before trusting this fix blindly: run `flatpak search --columns=name,description,application,version,remotes firefox` directly in a terminal on the Asahi/Fedora machine and confirm real tab-separated output comes back. If `flatpak search` returns nothing at all regardless of flags, that's a separate, pre-existing issue — usually stale appstream metadata — fixable with `flatpak update --appstream`. Note in a code comment which case applies once confirmed, so this doesn't get "fixed" again by mistake later.

Step 3: Tests

Mock subprocess output with realistic tab-separated columns (including a case with an empty remotes field) and assert parsing.

Task 4: Give PIPX a Real Search (Local Index Cache)

Files:

- Modify: `app/core/drivers/pipx.py`
- Create: `tests/test_pipx_search.py`

Step 1: Understand why the old approach can't be fixed in place

PyPI does not have a general-purpose search API — the old XML-RPC `search()` method was disabled in 2018 due to abuse, and there's no supported replacement. The only reliable "search" available is a substring match

against the full list of package names, which PyPI does expose cheaply via its Simple Index.

Step 2: Fetch and cache the Simple Index

```
_INDEX_CACHE_PATH = Path.home() / ".cache"
/ "polyget" / "pypi_simple_index.json"
_INDEX_MAX_AGE_SECONDS = 7 * 24 * 60 * 60
# 1 week

    async def _ensure_index_cached(self) ->
list[str]:
    """Download and cache the full list of
PyPI package names, refreshing weekly."""
    import time
    if self._INDEX_CACHE_PATH.exists():
        age = time.time() -
self._INDEX_CACHE_PATH.stat().st_mtime
        if age <
self._INDEX_MAX_AGE_SECONDS:
            return
    json.loads(self._INDEX_CACHE_PATH.read_text())

self._INDEX_CACHE_PATH.parent.mkdir(parents=True, exist_ok=True)
    proc = await
asyncio.create_subprocess_exec(
        "curl", "-s", "-H", "Accept:
application/vnd.pypi.simple.v1+json",
        "https://pypi.org/simple/",
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE
    )
    stdout, _ = await
    asyncio.wait_for(proc.communicate(),
```



```

timeout=30.0)
    data =
    json.loads(stdout.decode(errors="ignore"))
    names = [p["name"] for p in
data.get("projects", [])]

self._INDEX_CACHE_PATH.write_text(json.dumps(n
ames))

    return names

```

Step 3: Implement `search_packages()` as a local substring match, then fetch descriptions only for the top N

```

    async def search_packages(self, query:
str) -> list[dict[str, Any]]:
        try:
            names = await
self._ensure_index_cached()
            query_lower = query.lower()
            matches = [n for n in names if
query_lower in n.lower()][:20]

            results = []
            for name in matches:
                # Fetch description only for
what we're actually going to show
                proc = await
asyncio.create_subprocess_exec(
                    "curl", "-s",
f"https://pypi.org/pypi/{name}/json",

                stdout=asyncio.subprocess.PIPE,
                stderr=asyncio.subprocess.PIPE
            )

```

```

        stdout, _ = await
        asyncio.wait_for(proc.communicate(),
        timeout=5.0)
        info =
        json.loads(stdout.decode(errors="ignore")).get
        ("info", {})
        results.append({
            "name": name, "id": name,
            "description":
        info.get("summary", ""),
            "version":
        info.get("version", "")
        })
        return results
    except Exception:
        return []

```

Note the tradeoff explicitly: this does up to 20 follow-up requests per search (one per matched name) to get descriptions, which is slower than the other managers. If that proves too slow in practice, drop descriptions from initial results and only fetch one on-demand when the user selects an item, rather than for all 20 up front.

Step 4: Tests

Mock the index cache file with a small fixed list of names, assert substring matching works case-insensitively, and assert `_ensure_index_cached()` doesn't re-download when the cache file is fresh (mock `time.time()` and file mtime).

Task 5: Refactor `SearchWorker` to Loop Over

`discover_managers()`

Files:

- Modify: `app/ui/main_window.py`

Step 1: Replace the five hardcoded async functions with a generic loop

```
class SearchWorker(QThread):
    """Worker thread to run asynchronous
    searches across all managers that support
    it."""
    results_signal = Signal(list)
    log_signal = Signal(str)

    def __init__(self, query: str,
source_filter: str = "All", parent: Any =
None):
        super().__init__(parent)
        self.query = query
        self.source_filter = source_filter

    def run(self):
        loop = asyncio.new_event_loop()
        asyncio.set_event_loop(loop)
        results = []

        async def search_one(mgr:
PackageManager):
            if self.source_filter not in
("All", mgr.name):
                return
            try:
                items = await
```

```

mgr.search_packages(self.query)
    for item in items:
        item.setdefault("source",
mgr.name)
        results.append(item)
    except NotImplementedError:
        pass
    except Exception as e:
        self.log_signal.emit(f"
{mgr.name} search error: {str(e)}")

    managers = discover_managers()

loop.run_until_complete(asyncio.gather(*
(search_one(m) for m in managers)))
loop.close()
self.results_signal.emit(results)

```

Step 2: Verify the source filter dropdown

(combo_source) still populates correctly

It currently likely hardcodes the five manager names — check where `combo_source` items are added and switch that to `[m.name for m in discover_managers()]` too, so a newly catalog-added manager with search support appears there automatically.

Step 3: Regression test

Manually run a search with each currently-installed manager individually selected in the source filter, plus "All", and confirm results still populate as before the refactor — this task changes *how* search runs but should produce identical results for DNF/Cargo/NPM once Tasks 2–4 are done.

Task 6: Wrap-up

Files:

- Modify: `tests/test_drivers_installed.py`

Add one assertion that every manager either implements `search_packages()` (no `NotImplementedError`) or is documented as search-unsupported, so a future driver added without search support doesn't silently disappear from the store without anyone noticing it was never wired up.